

Feel it, say it, show it: emotionally expressive Furhat

Andrea Carrión del Fresno
LT2319
Dialogue Systems 2

1 Introduction

1.1 Motivation & Goal

My motivation for this project comes from a genuine interest in conversational social robots and a desire to understand how to make their interactions feel more human and expressive. After experimenting with integrating an Large Language Model (LLM) into a robot in earlier coursework, it became clear that the interaction felt flat and far from resembling a natural conversation.

An LLM’s output is purely text-based and it is not inherently designed for multimodal generation (at least not in the way an embodied conversational agent requires). When integrating an LLM into a social robot, we risk losing a crucial part of human communication if we rely only on spoken text. Without facial expressions or prosodic cues, much of the communicative richness that characterizes natural interaction disappears.

This made me curious about how an LLM’s textual output could be adapted into spoken, expressive behavior by combining language, voice prosody, and non-verbal gestures in a coherent emotional way. Furhat, with its flexible expressive capabilities, is ideal for exploring this challenge.

The goal of this project is therefore to explore how to bridge the gap between LLM-generated text and expressive non-verbal behavior on Furhat. My approach is to prompt the LLM to explicitly output an emotional tone that matches its response. This emotional tone can then be mapped to

custom-made Furhat gestures and to specific voice styles, allowing the robot to "show" and "sound" the emotion behind the words.

2 Services & APIs

This project relies on three core components: the LLM prompting pipeline, gesture generation, and voice synthesis. These are implemented through a combination of external APIs and pre-existing services, including the Furhat Remote API, Ollama's LLM API, and Furhat's built-in TTS integrations (Microsoft Azure and Amazon Polly).

All robot actions (i.e., speech, gestures, user tracking, etc) are executed using Furhat Remote API. Furhat exposes REST endpoints such as `/furhat/say`, `/furhat/listen`, `/furhat/gesture`, `/furhat/attend`, `/furhat/voice`, etc. These endpoints allow the XState machine to fully control the robot from outside the Furhat system. Custom gestures were defined manually in TypeScript and sent to Furhat as JSON gesture definitions through the API Section 3.2 provides further details on gestures).

To generate dynamic conversational responses, the project uses Ollama, an open-source tool for running LLMs locally. The chosen model is llama3.1, accessed via Ollama's HTTP API.

Furhat supports several speech synthesis providers, including Microsoft Azure, Amazon Polly, Acapela, and ElevenLabs. I decided to work with Azure and Polly because Furhat Robotics provides built-in API credentials for these providers, which means the user does not need to authenticate or call these services directly. Instead, Furhat exposes all supported voices through its own REST endpoint (e.g., `/furhat/voice?name=AriaNeural`).

3 System Components

3.1 LLM Prompting Pipeline

The first core component of the system is the prompting pipeline used to communicate with the LLM. The goal of this module is to take the user's input, pass it to the LLM along with a set of instructions, and obtain a structured response. This is implemented inside the *LLMActor*, using the *system* role.

The instructions specify how the model should behave. They also indicate that the response should include one of several provided emotional tones (i.e., *cheerful*, *sad*, *angry*, etc) and that it must be returned as a JSON object of the following form:

```
{
  "response": "<natural language response text>",
  "voiceStyle": "<chosen emotional tone>"
}
```

This structure allows the rest of the system to automatically map the emotional tone label (*voiceStyle*) to the corresponding gesture and voice style. An example output would be:

```
{
  "response": "Hello there! It's lovely to meet you!",
  "voiceStyle": "friendly"
}
```

Note that the list of emotional tones included in the prompt were not arbitrary, but they were specifically selected because they match the style tags supported by Microsoft Azure neural voices¹ (i.e., *en-US-AriaNeural* supports *cheerful*, *sad*, *angry*, *excited*, etc).

Since the intention was to generate a wide range of expressive behaviors, I experimented with multiple versions of the system prompt. By testing these variations, I compared how each prompt affected the model's emotional range, output structure and consistency.

Approach 1: Baseline instruction (instructionsLLM)

This first prompt defines the assistant as an *expressive AI assistant* and asks it to choose the *voiceStyle* that best matches its reply.

```
instructionsLLM: ‘
  You are an expressive AI assistant controlling a social robot.
  Each time you respond, return your answer as a JSON object
```

¹<https://learn.microsoft.com/en-us/azure/ai-services/speech-service/language-support?tabs=tts#voice-styles-and-roles>

formatted like this:

```
{
  "response": "<your natural language response text>",
  "voiceStyle": "<one of: ${voiceStylesList.join(", ")}>"
}
```

Choose the voiceStyle that best matches the emotional tone of your response.

Only respond with the JSON, no explanations or text outside the object.

```
'.trim(),
```

The LLM responses were mostly *friendly* or *cheerful*, which is somehow expected because they tend to optimize for friendliness unless strongly prompted otherwise.

Approach 2: Explicit emotional guidelines (instructionsLLM2)

The second version adds more detailed emotional guidelines and explicitly asks the LLM to avoid using friendly tones all the time:

instructionsLLM2: ‘

You are an emotionally expressive AI assistant controlling a social robot. Your job is to respond to the user in natural, emotionally aware dialogue.

Each time you respond, you MUST:

1. Choose an emotional tone (voiceStyle) that best matches the user’s message or context.
2. Vary your tone naturally - do NOT always sound friendly or cheerful. Your goal is to show a wide emotional range across the conversation.

Available voice styles:

[cheerful, sad, angry, calm, friendly, empathetic, serious, excited, depressed, hopeful]

Guidelines:

- Use "cheerful" for greetings, jokes, or good news.
- Use "sad" or "depressed" when the user shares loss, disappointment, or pain.
- Use "angry" if the user mentions unfairness, injustice, or frustration.
- Use "calm" to help relax the user or de-escalate tension.
- Use "friendly" for small talk and casual interactions.
- Use "empathetic" when the user expresses vulnerability, worry, or fear.
- Use "serious" when discussing facts, ethics, or sensitive social issues.
- Use "excited" when the user shares achievements or surprising good news.
- Use "hopeful" when talking about improvement, recovery, or optimism.

Return ONLY a JSON object like this:

```
{
  "response": "<your natural language response text>",
  "voiceStyle": "<one of the styles above>"
}
```

Do not include any explanations, quotes, or text outside this JSON.

```
`trim(),
```

Unfortunately, this version did not significantly increased emotional diversity in the LLM's responses. It also occasionally produced malformed JSON objects, causing parsing issues.

Approach 3: Performing actor (instructionsLLM3)

The third version instructs the model to behave like a performing actor who adjusts their emotion according to the user's message. In this case the model is told to interpret the user's input rather than match its own response to an emotion. This aims to shift the LLM away from its default polite/friendly assistant mode.

```
instructionsLLM3: `
```

You are an expressive AI actor controlling a social robot.
Treat every user message as if it were an acting cue or emotional direction.
Your job is to perform short, emotionally expressive responses that reflect the mood or situation described by the user.

Each time you respond, do the following:

1. Decide which emotional tone best fits the user’s cue or message.
2. Choose one of the available styles: [cheerful, sad, angry, calm, friendly, empathetic, serious, excited, depressed, hopeful].
3. Return your response as a JSON object only, like this:

```
{  
  "response": "<your short expressive line>",  
  "voiceStyle": "<chosen emotion>"  
}
```

Keep the response short and natural, like an actor performing one or two lines.

Never explain what you are doing - just perform the line.

```
‘.trim(),
```

Although this approach initially produced more varied emotional outputs, it went back to mostly "friendly" after a few turns. Additionally, the model sometimes included text outside the required JSON format, unlike it was instructed to do. Nevertheless, this approach seemed to influence the model’s behavior more strongly than the emotional-guideline version (approach 2).

Final choice

Even though the baseline prompt (approach 1) produced the least emotional variation, it also generated the most stable and well-formatted responses. Since the rest of the system components depend on the correct JSON formatting in order to work properly, the first approach was chosen as the final version, but keeping in mind that emotional diversity remains limited unless the user provides explicit emotional cues (i.e.: "Answer in an optimistic, hopeful tone").

3.2 Gesture Generation

The gestures used in this project are a mix of custom-made and built-in gestures. The custom-made gestures were designed to represent the different emotional tones that match the style tags supported by Microsoft Azure (as introduced in Section 3.1). These gestures are created by combining parameters (LOOK_DOWN_LEFT, BROW_UP_LEFT, NECK_TILT, etc) from Furhat’s BasicParams library². For example, combining SMILE_OPEN, EYE_SQUINT_LEFT, BROW_UP_LEFT, etc, helps create a cheerful expression. A total of 20 custom gestures were created, two for each emotional tone (e.g., *cheerfulGestureA*, *cheerfulGestureB*, *sadGestureA*, *sadGestureB*, etc). The actor *fhLLMGesture* calls the function *selectGestureByVoiceStyle()*, which takes the selected voice style (i.e., emotional tone) and randomly picks a corresponding gesture from the *emotionGestures* array.

In addition to these emotion-specific gestures, the system also uses a set of "universal gestures". These includes Furhat’s built-in gestures (*Blink*, *BrowRaise*, *Smile*) as well as three additional custom gestures (*Happy*, *DoubleNod*, *GazeAway*). Universal gestures can be used interchangeably at any moment while Furhat is listening to the user and are not tied to a particular emotional tone. Their purpose is to make the interaction feel more natural by creating the illusion that the robot is actively attending rather than remaining still. This behavior is handled by the *fhUniversalGestures* actor, which selects a random universal gesture each time the robot enters the listening state.

3.3 Voice Synthesis

As mentioned in Section 2, since Furhat Robotics provides built-in API credentials for Microsoft Azure and Amazon Polly, the project initially explored both of these speech synthesis providers.

While some Amazon Polly neural voices³ offer different styles, Furhat only supports *Neutral*, *Conversational* and *News*. In contrast, Microsoft Azure neural voices support a much wider range of expressive styles (*angry*, *cheerful*, *customerservice*, *empathetic*, *excited*, etc), which seemed a better choice for this project since we are seeking voice expressiveness.

²<https://docs.furhat.io/deprecated/remote-api#gesturedefinition>

³<https://docs.aws.amazon.com/polly/latest/dg/supportedtags.html>

Among Azure’s voices, *en-US-AriaNeural* supports the largest number of style tags, so it was selected as the base voice. The intended design was to modulate its expressive style according to the emotional tone returned by the LLM. For instance:

- If the LLM outputs *"voiceStyle": "sad"*, then *en-US-AriaNeural* should speak using the *sad* style.
- If the LLM outputs *"voiceStyle": "excited"*, the same voice (*en-US-AriaNeural*) should switch to the *excited* style.

This would ensure a consistent voice identity while still expressing a wide emotional range.

At this point, the first challenge arise: the Furhat documentation did not explain how to use Azure’s voice styles through the Remote API, using Typescript. While the documentation states that Furhat supports Microsoft Azure and Amazon Polly TTS, all examples involving voice styles are written in Kotlin and seem to relate to the Skill SDK rather than the Remote API. It was therefore unclear whether Azure style tags are actually supported in a way accessible from Typescript.

A second challenge involved setting the correct voice name. When using the *fhVoice()* function, the voice identifier in the format used by Azure’s official documentation was initially passed (e.g., *"en-US-AriaNeural"*). However, Furhat’s Remote API expects a different naming convention (e.g., *"AriaNeural"*), matching the labels shown in the Furhat web interface. Even though this was easy to fix, it still required trial and error because the documentation did not clarify it, which cost extra time.

An alternative strategy was to use SSML (Speech Synthesis Markup Language) tags, which are supported by Amazon Polly. This would have involved embedding the LLM’s response inside custom SSML tags, allowing control over speaking style, rate and pitch to match the each emotional tone. This approach is more time-consuming since it requires to manually design each tag. However, this strategy was ultimately discarded because Furhat’s Remote API endpoint */furhat/say?text=* does not interpret SSML. Instead, it treats the SSML-tagged string literally as plain text. In practice, this means the Remote API cannot handle SSML directly and that SSML is only available within Skill SDK.

Since neither Azure’s voice styles nor Polly’s SSML tags could be used, the final implementation maps each emotional tone to a different Azure voice

(e.g., *cheerful*: "*AvaNeural*", *sad*: "*AshleyNeural*", etc). This does not reflect authentic emotional variation, but it allows the demo video to present distinguishable vocal differences for each emotion.

Finally, voice synthesis was the most technically challenging component of the project. Multiple approaches were tested and, even though the current solution is simply functional for demonstration purposes, achieving expressive emotional speech in the Remote API remains an open area for future work.

4 Implementation

The system is implemented as a state machine that coordinates user detection, greeting, LLM calls, gesture generation, voice synthesis and conversation flow. A description of the behavior of each state and its actors is as follows:

Start → GetUser

The system begins in the *Start* state and transitions to *GetUser*. Here, the *fhGetUser* actor attempts to detect the user.

- onDone → transition to *Attend* state
- onError → transition to *Fail* state

This *onDone/onError* fallback logic appears throughout the state machine (*Attend*, *Greeting*, *LLMSpeaks*, etc).

Attend

In *Attend*, the *fhAttend* actor makes Furhat look at the user's position so that the interaction begins with proper attention.

- onDone → transition to *Greeting* state

Greeting

This state triggers the first interaction with the LLM. Before invoking the model, the state sets an initial messages list containing the system prompt

(*instructionsLLM*) and a predefined user message instructing the LLM to start the conversation with a short greeting. Then the state invokes *LLMActor*, which returns a JSON object containing the LLM’s response and its selected emotional tone. On success, the system saves the LLM’s answer (*llmResponse*), the selected emotional tone (*llmVoiceStyle*) and appends the message to the running dialogue (*messages*). It later transitions to *LLMSpeaks*.

LLMSpeaks

This state handles the robot’s multimodal expression of the LLM’s answer, that is, the corresponding gesture and voice. There are two child states:

Gesture: *fhLLMGesture* selects and performs one of the gestures linked to the received emotional tone (e.g., *friendlyGestureB*, *sadGestureA*, etc).

Talking: *fhLLMSpeak* synthesizes the LLM’s answer using the corresponding Furhat voice.

An example output could be:

```
Response: Hello there! It's lovely to meet you!
Selected gesture: friendlyGestureB
Selected voice: EmmaNeural
```

- onDone → transition to *Listen* state

Listen

This state listens to the user input while keeping Furhat expressive and responsive. It has two child states:

UniversalGestures: *fhUniversalGestures* triggers a random universal gesture to make Furhat appear attentive while listening.

Listening: The *fhL* actor starts the speech recognizer. When the user speaks (onDone), it saves the captured text as *userInput* and appends it to *messages* as a new user entry. Then, it transitions to *TestLLM*.

TestLLM

This state evaluates the user’s input and decides how the interaction continues.

If the recognized text contains any stopword (listed in *stopWords*), the system transitions to *Goodbye* as it means the user wants to end the conversation. Otherwise, the LLM is invoked again with the updated messages list. On success, the system saves the new LLM response and emotional tone, it appends the assistant reply to *messages*, and lastly returns to *LLMSpeaks*. This creates the main interaction loop: *LLMSpeaks* $\rightarrow$ *Listen* $\rightarrow$ *TestLLM* $\rightarrow$ *LLMSpeaks* $\rightarrow$ [...]

The recursion continues until the user ends the conversation or an error breaks the loop.

Goodbye

In this state, *fhTalkOriginal* generates a goodbye message spoken with Furhat’s default voice.

- onDone $\rightarrow$ transition to *Kiss* state

Kiss

fhKissing plays a kiss gesture and sound effect to close the interaction in a playful way.

- onDone $\rightarrow$ transition to *End* state

End

This is the machine’s final state which prints a last message (“End of the conversation”).

5 Demo

To demonstrate how the system is intended to behave in an ideal interaction, a demo video⁴ was created showing Furhat performing expressive gestures and switching between different voice styles.

⁴<https://youtu.be/6dGcileuoZE>

The demo video presents a scripted conversation designed to showcase the system’s full range of expressive behaviors. Because it is difficult to reliably trigger every gesture and voice style in a single live interaction with Furhat, the video does not invoke the LLM in real time. Instead, it follows a predefined dialogue that intentionally covers all ten emotional tones used in the project: *cheerful*, *sad*, *angry*, *calm*, *friendly*, *empathetic*, *serious*, *excited*, *depressed* and *hopeful*.

Although the interaction is scripted, the robot’s spoken lines are actual LLM responses obtained during earlier test sessions. The purpose of the video is therefore to show the complete collection of custom gestures and Azure voices in a controlled way, ensuring that each expressive behavior can be clearly observed.

6 Discussion

The prompt used to instruct the LLM is crucial to this project. While prompting is (relatively) simple, fast and requires very little code, it also introduces a few limitations. As discussed in Section 3.1, LLMs tend to default to socially polite behavior, which often results in "friendly" or "cheerful" responses unless explicitly told otherwise.

The choice of using the "system" role in *LLMActor* combined with instructions that frame the model as an "AI assistant" may cause this friendly bias. This is an interesting aspect to investigate in the future.

Additionally, it seems that the prompt alone is not enough to elicit diverse emotional tones unless the user input itself frames the emotion explicitly (e.g., "Pretend you’re an actor performing a high-tension movie scene. You’re really *angry*."). Without such cues, emotional variation remains limited.

Moreover, obtaining a consistent JSON-formatted response from the LLM becomes increasingly challenging as the interaction continues (typically after around eight turns). Below is a failure output example where the model outputs a natural language line outside the JSON object:

```
"Alright, let's heat it up! We can start with the basics: the
step-touch and the hip action."
{
"response": "We can start with the basics: the step-touch and
the hip action.",
```

```
"voiceStyle": "friendly"  
}
```

Lastly, another important point to discuss is whether the prompts should be topic-specific. In the current implementation, the instructions do not define a particular domain or scenario. In other words, the LLM simply responds to whatever the user says. It is the user who decides on the topic of the conversation and leads it. It may be worth exploring whether more domain-specific prompts lead to more varied emotional cues, consistent formatting and an overall output quality.

7 Future Work

This project was constrained by both time and several technical challenges, which naturally opens the door to future improvements. One potential direction is to leverage the prompting strategy or explore alternative approaches that support emotional variation. General-purpose LLMs tend to default to friendly tones unless strongly guided, so improving this aspect could lead to more expressive behavior.

Moreover, gesture design could also benefit from improvement. Although the project includes a diverse set of custom gestures, even more realistic behavior could be achieved by researching on human gesture production, body language or even cultural variation.

Voice expressivity is the biggest limitation of this project. Finding a way to employ Azure’s built-in style tags or SSML-style emotional cues through the Furhat Remote API was a major challenge that remains unsolved. Thus, exploring other alternatives such as using Furhat Skills may make it possible to apply emotional voice styles directly, which would significantly enhance expressiveness.

Finally, once these improvements are in place, a natural next step would be to conduct a user study. Observing real users interacting with Furhat would provide valuable insights into how effectively the system communicates emotion and how the multimodal behaviors influence the overall interaction experience.